Linux Hardening Auditor

A CIS-mapped audit & safe-remediation tool for Ubuntu 22.04

Project Closeout Report

Milad Bahari Qaragoz

24 June 2026

1 Executive summary

The Linux Hardening Auditor is a Python tool that audits a Linux host against the **CIS Ubuntu 22.04 LTS Benchmark v1.0.0** (with BSI IT-Grundschutz alignment where relevant), reports pass/fail per control with severity and remediation, and can optionally and *reversibly* fix what it finds. It is read-only by default; every mutating action backs up the file it touches first and re-verifies the control afterwards.

The tool was validated end-to-end on a stock **Ubuntu 22.04.5 LTS** cloud host. A default image scored **53 %**; after a single `fix` run it scored **100 %**, with all 15 remediable controls remediated *and* independently re-verified against the live host.

Stage	Score	Controls passing
Before — default Ubuntu 22.04 image	**53 %**	17 / 32
After — single `fix` run, re-audited	**100 %**	32 / 32

2 Design principles

The codebase is judged against three non-negotiable principles, chosen so the project reads like the work of someone who builds auditable, maintainable security tooling:

- **Scalable.** Controls are data-driven, self-registering plugins. Adding control #33 means writing one self-contained check module and nothing else — the engine, reporters, and remediator never grow per control.
- **Traceable.** Every control maps to a real CIS control ID. A reader can go from a CV bullet → report line → control ID → the exact check code → the written rationale, with no missing link. Six Architecture Decision Records capture *why*, not just what.
- **Debuggable.** Structured logging at every check, deterministic output ordered by control ID, dry-run by default, and clear "expected vs. found" failure messages.

3 Architecture

The engine discovers and runs whatever checks are registered and emits a uniform result that the reporters render; it never names a control.

```
auditor/
  cli.py          # entrypoint: audit | fix
  engine.py       # discovers + runs checks, collects results
  registry.py     # control + fixer discovery (@control / @fixer)
  models.py       # Control / CheckResult / Severity / Status
```

```
host.py          # injected read-only host (LocalHost / FakeHost)
reporters/       # console - markdown - html - json
remediation.py   # backup -> dry-run -> apply -> verify
checks/          # ONE self-registering module per control
```

Adding a control is a five-step, single-file path: create `checks/<id>_<slug>.py`, let it self-register via the `@control` decorator, add a catalogue row and a threat-model rationale, add a test, add a changelog line. The host is injected, so every check is unit-testable on a non-Linux dev box through a `FakeHost`.

4 Control catalogue

The catalogue covers **32 controls** spanning SSH hardening, kernel `sysctl` network hardening, file permissions, password policy, the host firewall, auditing, and automatic security updates.

Severity	Count	Examples
High	7	SSH `PermitRootLogin no`, host firewall active, auditd enabled, `/etc/shadow` mode
Medium	23	`sysctl` network hardening, SSH MACs/KEX/MaxAuthTries, password min-length & expiry
Low	2	cron daemon active, password-expiry warning age
Total: 32		

5 Remediation engine

Remediation is split into three independently testable parts (ADR-0005):

1. A control's **fixer** is a pure function `fix(host) -> [Action]`; it only *reads* the host and returns a declarative plan, so `-dry-run` is "plan, then print" with zero side effects.
2. An **Applier** performs the side effects, **backing up every file before it changes it**. SSH changes are written as drop-in files and validated with `sshd -t` *before* the daemon is reloaded, so a bad config can never lock you out.
3. After a control's actions are applied, the control's **own check is re-run to verify** — a fix that does not make the check pass is reported as failed, not assumed.

5.1 The shared-file bug and its fix (ADR-0006)

Two controls legitimately edit the same file: `5.5.1.1` (`PASS_MAX_DAYS`) and `5.5.1.2` (`PASS_MIN_DAYS`) both rewrite `/etc/login.defs`. Because each fixer captured the file's content *at plan time*, applying them in sequence let the second whole-file write silently clobber the first fixer's change — only the last writer of a shared file survived. This surfaced during the cloud demo run.

Fix: `apply` now re-derives each control's actions by calling its fixer again against the *live* host at apply time, so every fixer sees the previous one's edits. The backup layer additionally keeps the *first* backup of a file touched twice in one run, so the saved copy is the true pre-run original. The up-front plan still drives the read-only `-dry-run` preview. Both the shared-file case and the command-failure abort path are covered by regression tests.

6 Real-host verification

6.1 Method

The tool was run end-to-end on a freshly created **Ubuntu 22.04.5 LTS** Google Compute Engine VM (`e2-micro`, Python 3.10.12) — the exact distribution and interpreter the benchmark targets. The audit read live `sshd -T` output, `/proc/sys`, `/etc/login.defs`, and real file modes; it was reached over an IAP tunnel with no public IP, and given outbound-only internet via Cloud NAT so package-installing fixers could run.

6.2 Before — default image (53 %, 17/32)

```
Linux Hardening Audit -- lha-audit-vm -- 2026-06-24T08:56:21
Score: 53%  (17 / 32 controls passing)

HIGH
  [FAIL] 3.5.1.3  Host firewall active        found "ufw inactive"
  [FAIL] 4.1.1.2  auditd enabled and active   found "active=inactive"
  [FAIL] 5.2.7    SSH PermitRootLogin disabled found "without-password"
MEDIUM
  [FAIL] 1.5.4    Core dumps restricted        fs.suid_dumpable=2 (want 0)
  [FAIL] 3.3      Kernel network hardening     1 of 7 not hardened
  [FAIL] 5.2.12   SSH X11Forwarding disabled   found "yes"
  [FAIL] 5.2.14   SSH strong MACs only         weak MACs present
  [FAIL] 5.2.16   SSH AllowTcpForwarding off   found "yes"
  [FAIL] 5.2.18   SSH MaxAuthTries <= 4        found "6"
  [FAIL] 5.2.21   SSH LoginGraceTime <= 60s    found "120"
  [FAIL] 5.4.1    Password min length >= 14    pwquality.conf missing
  [FAIL] 5.5.1.1  Password expiration <= 365   PASS_MAX_DAYS=99999
  [FAIL] 5.5.1.2  Min days between changes >=1 PASS_MIN_DAYS=0
  ... (15 failing in total)
```

6.3 Apply — 15 controls remediated and verified

```
Remediation applied (backups in backups/20260624-085815/):
  [OK] 1.5.4   Core dumps restricted          -- now passing
  [OK] 3.3     Kernel network hardening        -- now passing
  [OK] 3.5.1.3 Host firewall active            -- now passing
  [OK] 4.1.1.2 auditd enabled and active       -- now passing
  [OK] 5.1.2   /etc/crontab permissions        -- now passing
  [OK] 5.2.1   sshd_config permissions         -- now passing
  [OK] 5.2.7   SSH PermitRootLogin disabled    -- now passing
  [OK] 5.2.12  SSH X11Forwarding disabled      -- now passing
  [OK] 5.2.14  SSH strong MACs only            -- now passing
  [OK] 5.2.16  SSH AllowTcpForwarding disabled -- now passing
  [OK] 5.2.18  SSH MaxAuthTries <= 4           -- now passing
  [OK] 5.2.21  SSH LoginGraceTime <= 60s       -- now passing
  [OK] 5.4.1   Password minimum length >= 14   -- now passing
  [OK] 5.5.1.1 Password expiration <= 365 days -- now passing
  [OK] 5.5.1.2 Min days between changes >= 1   -- now passing
15/15 control(s) remediated and verified.
```

The two `/etc/login.defs` controls (`5.5.1.1` and `5.5.1.2`) **both** verify, and `login.defs` appears in the backup tree exactly once — confirming the ADR-0006 fix on a real host. After the run, a fresh read-only audit reported **100 % (32/32)**.

7 Quality, testing & reproducibility

- **147 automated tests** (`pytest`), all OS-independent via `FakeHost`, covering each check, the engine, the registry, every reporter, and the remediation plan/apply/verify loop — including the shared-file regression.
- **Deterministic output** ordered by control ID, so report diffs are meaningful.
- **Standard library only** at runtime; `ruff` for lint/format; Python 3.10+ to match Ubuntu 22.04's default interpreter.
- **Maintained documentation discipline:** every behaviour change updates `README`, `CHANGELOG`, the decision log, the control catalogue, and the threat model in the same commit.

8 Project status

Closed. The tool audits 32 CIS-mapped controls, remediates the 15 that are safely auto-fixable, and has been verified before/after on a real Ubuntu 22.04 host (53 % → 100 %). Possible future work, none blocking: cross-checking results against Lynis and OpenSCAP, and broadening the catalogue beyond the current 32 controls.

Run it:

```
python -m auditor audit              # read-only audit, console summary
python -m auditor audit --report md  # write a Markdown report
python -m auditor fix --dry-run      # preview changes, touch nothing
python -m auditor fix                # apply safe fixes, with backups
```

Generated as the closeout artifact for the Linux Hardening Auditor portfolio project. Source: `docs/closeout-report.tex`.